COMPSYS704 2017 LAB2
Compiling and running SystemJ programs

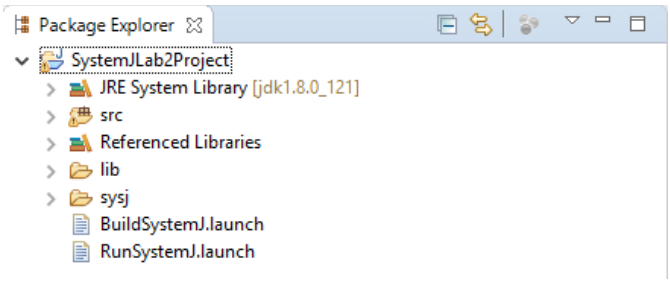
13/07/2023

The purpose of this lab is to familiarize you with the basics of the SystemJ workflow and writing/running SystemJ programs. In this lab, you are assumed to be compiling and running the programs using Eclipse IDE.

Prerequisite: The ability to import Eclipse projects as introduced in Lab 1.

1 Installing SystemJ tools

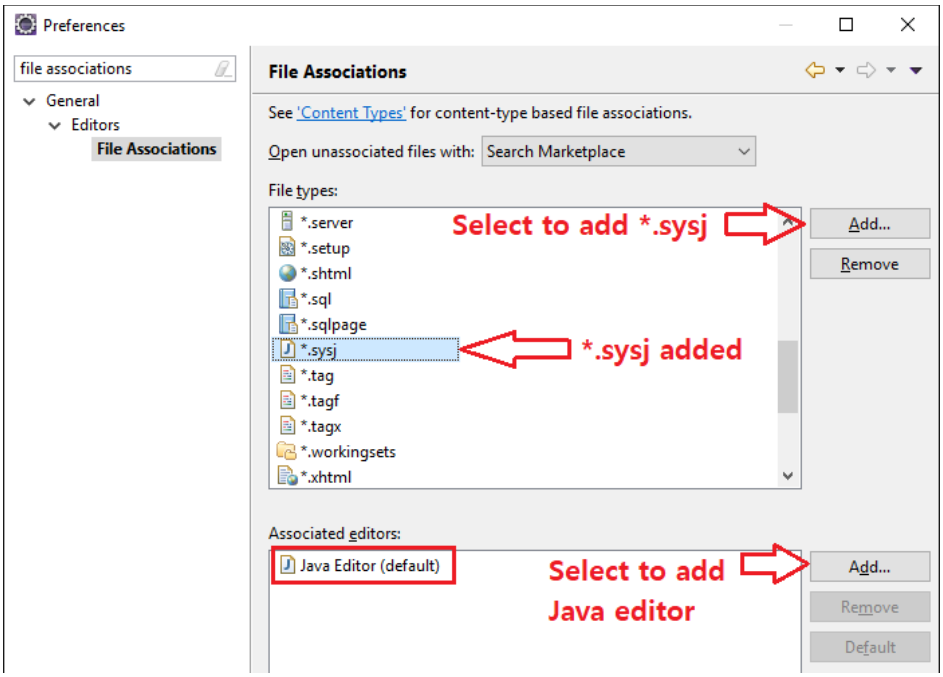
Download the SystemJ tool named **'COMPSYS704_Lab_2_New.zip**' from Canvas. This archive file contains an Eclipse project as well as a template SystemJ source codes used in this lab. Extract the archive and import the project to your Eclipse workspace. You will then see project structure as shown below.



where

1. **src** – A source directory where you create/put all Java source codes including the ones used in the SystemJ program.
2. **lib** – Contains SystemJ compiler and runtime.
3. **sysj** – A directory where you put all the SystemJ source codes and Local Configuration Files (LCFs). More about LCF will be introduced later.
4. ***.launch** files – These are Eclipse launch configuration files, which are used for compiling and running SystemJ programs.

It would be good to have some syntax highlighting and auto-completion (e.g. matching brackets) features when we develop SystemJ programs. While there is no official up-to-date SystemJ Eclipse plugin available, you can (for now) associate SystemJ files with a Java editor used in Eclipse. Select *Window > Preference* which will open a preference manager for the current eclipse project. Navigate to *General > Editors > File Associations* in the preference manager and add filetype *.sysj and associate it with the Java editor as shown below.



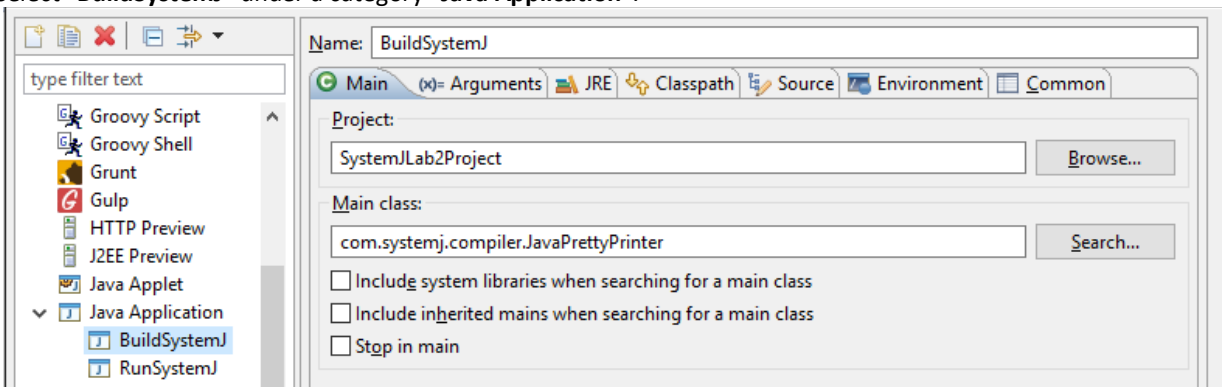
Now you are ready to develop SystemJ programs!

2 Running a simple program

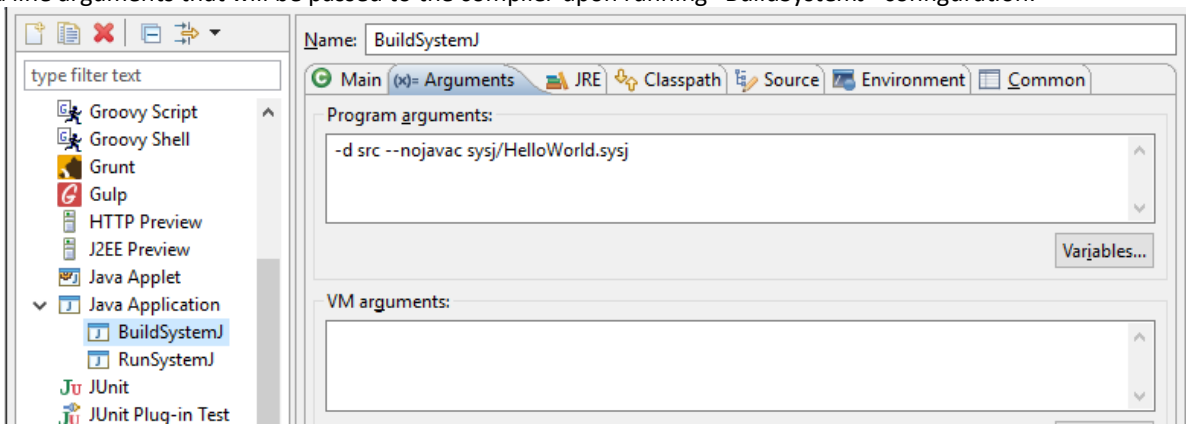
First, we are going to demonstrate how to build and run a very simple SystemJ program. Open **HelloWorld.sysj** in the **sysj** directory in the project. In this program, there is a single clock-domain called 'CD' which terminates after printing 'Hello World' to a console.

```
CD->{ System.out.println("Hello World"); }
```

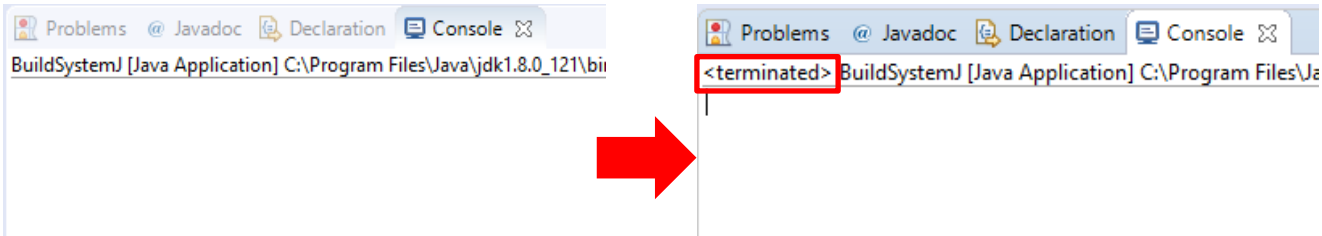
To build this program, select *Run -> Run configurations…* on the menu that will bring up a Run Configuration dialog as shown below. Select "**BuildSystemJ**" under a category "**Java Application**".



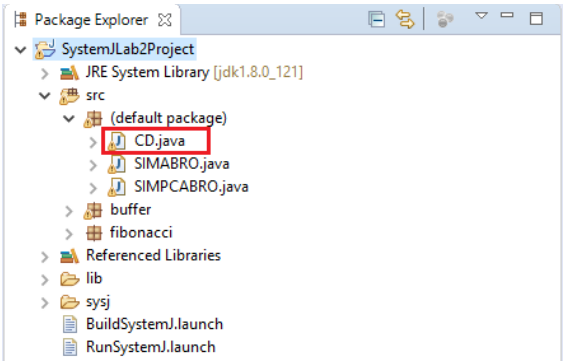
You will see it has already been configured and ready to run. If you are familiar with Java, you can easily see that the Main class of the SystemJ compiler is com.systemj.compiler.JavaPrettyPrinter. Click on the "Arguments" tab on the main where you can see command line arguments that will be passed to the compiler upon running "BuildSystemJ" configuration.



The option **-d** tells the compiler any generated files (.java) will be put into the "src" directory. The **--nojavac** option will delegate the compilation task for generating the final .class files to the Eclipse internal Java compiler. Lastly, the last argument is the target SystemJ code, **src/HelloWorld.sysj**, which we are compiling. Click "**Run**", the compilation is successful if you see the console state "<terminated>" after a few seconds without any error produced, see below.



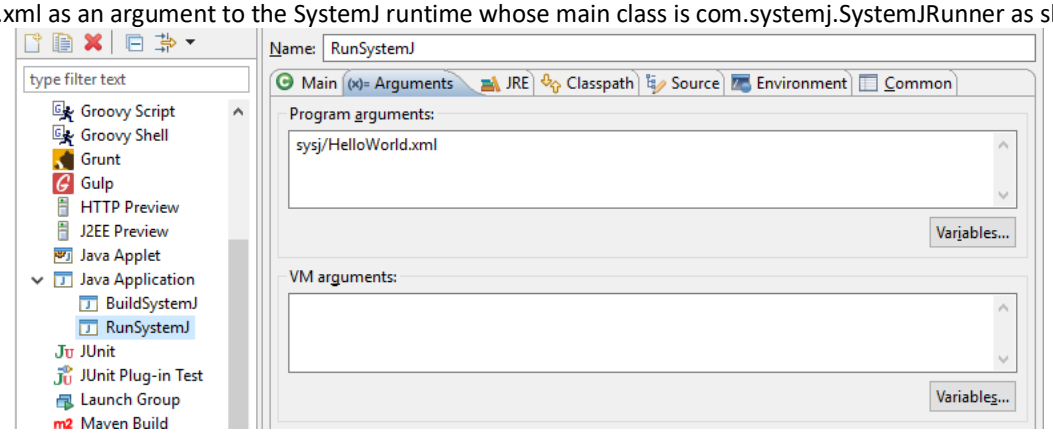
Note that the SystemJ compiler is a source-to-source compiler that converts .sysj files to one or more .java files. So we need the Java compiler (javac) provided by the Java Development Kit to generate the final .class files. Thankfully, Eclipse ships with its own Java compiler that automatically builds any Java files in the src folder, so we bypass javac by providing --nojavac option. Run "BuildSystemJ" that will generate CD.java in the src directory. Now select "SystemJLab2Project" in the Package Explorer and click "**F5**" to refresh. This step is important because Eclipse is not aware of CD.java that is generated by the SystemJ compiler so it will not build any generated Java files until it is refreshed manually by the user. After refreshing the project, you will see CD.java appears in the src directory as shown below.



In order to run the program, you need to invoke the SystemJ runtime system (RTS) (just like the Java runtime environment). An input to the RTS is a '**configuration file**' (LCF), written in a type of XML. This contains information about which CDs to run, how to map the I/O signals etc. Open the file '**HelloWorld.xml**' in the **sysj** directory and you will see the following.

```
<System xmlns="http://systemjtechnology.com">
 <SubSystem Name="Lab2" Local="true">
  <ClockDomain Name="MyCD" Class="CD"/>
 </SubSystem>
</System>
```

This LCF specifies that you are going to create and run a **subsystem** called 'Lab2'. *Note that unless you are setting 'Local' attribute to true, RTS will not run this subsystem.* Lab2 subsystem has a single clock-domain called 'MyCD' whose implementation is given as a fully qualified java class name (i.e. value of the 'Class' attribute). Finally, you can run the program by selecting "**Run SystemJ**" in the Run Configuration window (*Run -> Run Configurations…*). We are providing the LCF file src/HelloWorld.xml as an argument to the SystemJ runtime whose main class is com.systemj.SystemJRunner as shown below.



NOTE: Since compiling a SystemJ program involves two steps; (1) converting .sysj to .java and (2) generating .class files, error checking is also split into two steps. The SystemJ compiler first checks if there is any syntax error in the SystemJ source code whereas the Java compiler resolves class dependencies in the generated Java files. This sometimes results in the SystemJ compiler succesfully generating .java files but subsequent Java compilation errors in Eclipse (e.g. x icon in the src directory). In this case, you need to find out the causes for the errors (e.g. by inspecting the SystemJ source code or the generated Java file) and fix them. Examples of most common Java compilation errors are: incorrect variable names and missing Java packages/class files.

2.1 Using SystemJ statements

Now you know how to write and run SystemJ programs. In the following example, you will see how SystemJ components communicate with each other. Modify HelloWorld.sysj as follows:

```
CD->{
 signal A;
 await(A);
 System.out.println("Got a");
}
```

In the above example, the program blocks until signal A becomes true. However, since it is not emitted from anywhere, the program blocks at `await(A)` forever.

Exercise 1

Try to compile and run the program to see it yourself!

Open **Exercise-1.sysj**. It consists of two synchronous reactions:

```
CD->{
 signal A;
 { // First reaction
  await(A);
  System.out.println("Got a");
 }
 ||
 { // Second reaction
  emit A;
 }
}
```

The second reaction emits the signal A. As a result, the first reaction captures the signal and prints the message in the console. Try to compile and run the above program. **IMPORTANT**: You need to update the "**Argument**" option in the Run Configurations "**BuildSystemJ**" and "**RunSystemJ**" to replace the target files with "**Exercise-1.sysj**" and "**Exercise-1.xml**", respectively. Don't forget to refresh the project (by using F5) after compilation.

Exercise 2

Replace `await(A)` in Exercise 1 using SystemJ kernel statements so that it shows the same execution behaviour (i.e. prints "Got a"). A complete list of SystemJ kernel statements is shown below:

Kernel Statement	Description
`p1;p2`	p1 and p2 in sequence
`pause`	Consumes a logical instant of time (a tick boundary)
`[input|output] [<type>] signal <name>`	Declare a pure or valued signal
`emit <name>[(<EXP>)]`	Emit a signal with a possible value
`while(true) {p}`	Temporal loop
`present(<EXP>) {p} else {q}`	If signal S is present do p1 else do p2
`[weak] abort([immediate] <EXP>) {p} [do q]`	Pre-empt if S is present and control-flow enters q
`[weak] suspend ([immediate] <EXP>) {p}`	Suspend for 1 tick if S is present
`trap (<ID>) {p} do {q}`	Software exception. Control-flow enters q if an exception T happens
`exit(<ID>)`	Throw a software exception
`{p1} || {p2}`	Run p1 and p2 in lock-step parallel
`jterm(p)`	Execute a Java statement
`[input|output] [type] channel <name>`	Declare an input or output channel
`#<name>`	Retrieve data from a valued signal or channel

SystemJ also provides a number of syntactic sugars to ease programming efforts. They are derived from one or more SystemJ kernel statements:

Derived Statement	Expansion
`await(<name>)`	`/* Hidden for the Exercise 2 */`
`sustain(<name>)`	`while(true){emit <name>; pause;}`
`halt`	`while(true) {pause;}`
`loop`	`while(true)`

Lastly, the following statements are also derived using the kernel statements for sending/receiving data over the channels and pausing execution of a reaction for a specified period, respectively.

Complex derived Statement	Description
`send <name>(<exp>)`	Send data over the channel
`receive <name>`	Receive data over the channel
`waitl(<N> (ms|s))`	Causes an executing reaction to block for <N> ms or s

3 Using I/O signals

ABRO

ABRO was first introduced by Berry [1]. The program waits for the presence of the external signals A and B in the SystemJ program. When both of them have happened, the program emits an O to the environment. This behaviour is reset and restarted every time the input R has happened. Otherwise, the program just waits for R to happen. A finite state machine (FSM) capturing this reactive behaviour is shown in Figure 3.1 and a sample trace is also shown in the same figure below the FSM.

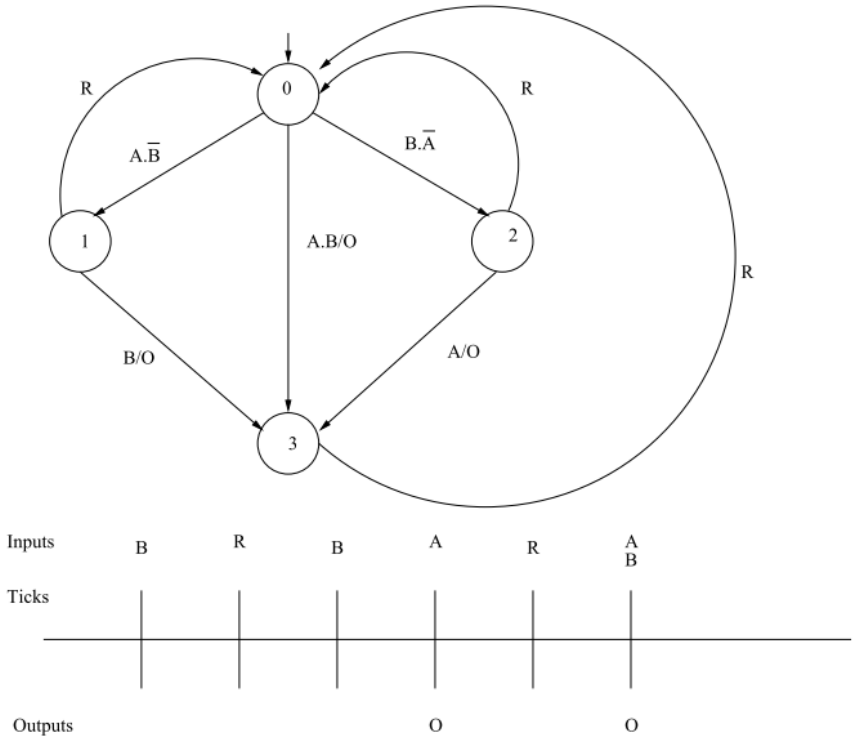


Figure 3.1 ABRO as FSM and one behavior trace

You are already given an ABRO clock-domain in **Exercise-3.sysj**. Try to read and understand the program logic. In this example, all signals [A, B, R, and O] are mapped to input and output signals. In particular, we are mapping these signals to the TCPSender and TCPReceiver classes so that input and output signals are exchanged with the external environment via TCP sockets as shown in **Exercise-3.xml**:

```
<System xmlns="http://systemjtechnology.com">
```

```
 <SubSystem Name="ABRO_SS" Local="true">
  <ClockDomain Name="CD1" Class="ABROCD">
   <iSignal Name="A" IP="127.0.0.1" Class="com.systemj.ipc.TCPReceiver"
Port="4000"/>
   <iSignal Name="B" IP="127.0.0.1" Class="com.systemj.ipc.TCPReceiver"
Port="4001"/>
   <iSignal Name="R" IP="127.0.0.1" Class="com.systemj.ipc.TCPReceiver"
Port="4002"/>
   <oSignal Name="O" IP="127.0.0.1" Class="com.systemj.ipc.TCPSender"
Port="5000"/>
  </ClockDomain>
 </SubSystem>
</System>
```

As you can see, input and output signals are specified using the elements '**iSignal**' and '**oSignal**' respectively. Each element has four attributes where:

- **Name** specifies the name of the signal which is being mapped
- **IP** specifies the IP address of the signal, used for creating server and client sockets in Java
- **Port** specifies the port number to be used. Note that only one TCPReceiver can listen to each port, so different ports must be used for each logical signal.
- **Class** specifies the fully qualified Java class name where the actual TCP/IP method is written

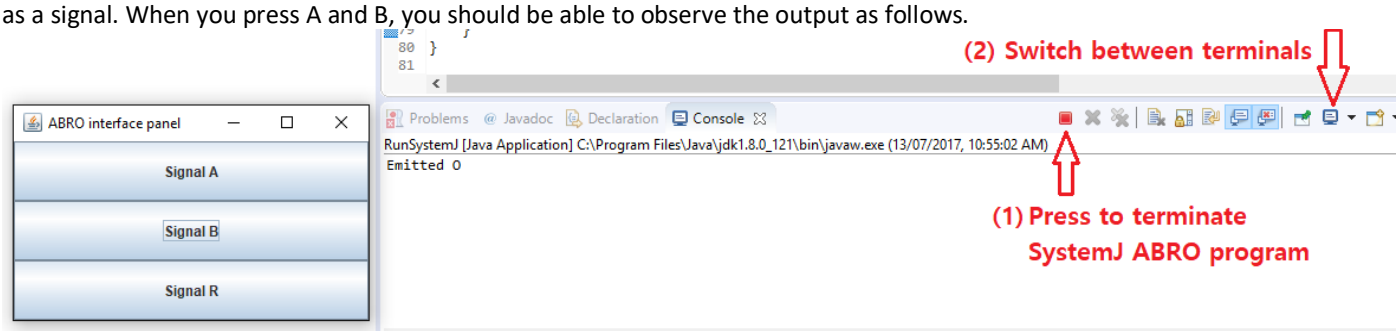
Therefore, **iSignal** instantiates an object called '**com.systemj.ipc.TCPReceiver**' and creates a TCP server using the given **IP** and **Port** in the LCF(XML). On the other hand, **oSignal** creates an instance of '**com.systemj.ipcTCPSender**' which is effectively a TCP client. Compile Exercise-3.sysj and run the program using Exercise-3.xml.

By default, TCPReceiver and TCPSender exchange a signal in the form of ***an array of Java objects*** where

- The first element of the array (object[0]) indicates the signal's status (type Boolean).
- The second element of the array (object[1]) is the value of the signal (any Java Object).

A simple Java program called '**SIMABRO.java**' is already given to you, which serializes and transmits a Java object using a socket. While the ABRO is running, compile and run **SIMABRO** as a separate Java Application.

When you press one of the buttons, the program transmits the Java object through TCP/IP, which will be received by one of the signal TCPReceivers (i.e. TCP servers instantiated in the SystemJ RTS) and then forwarded to the corresponding SystemJ program as a signal. When you press A and B, you should be able to observe the output as follows.



You can press a red square button (1) to terminate the ABRO program. When you run multiple SystemJ programs within the same Eclipse IDE, you can switch to different terminals associated with each program by pressing the icon (2).

Exercise 3

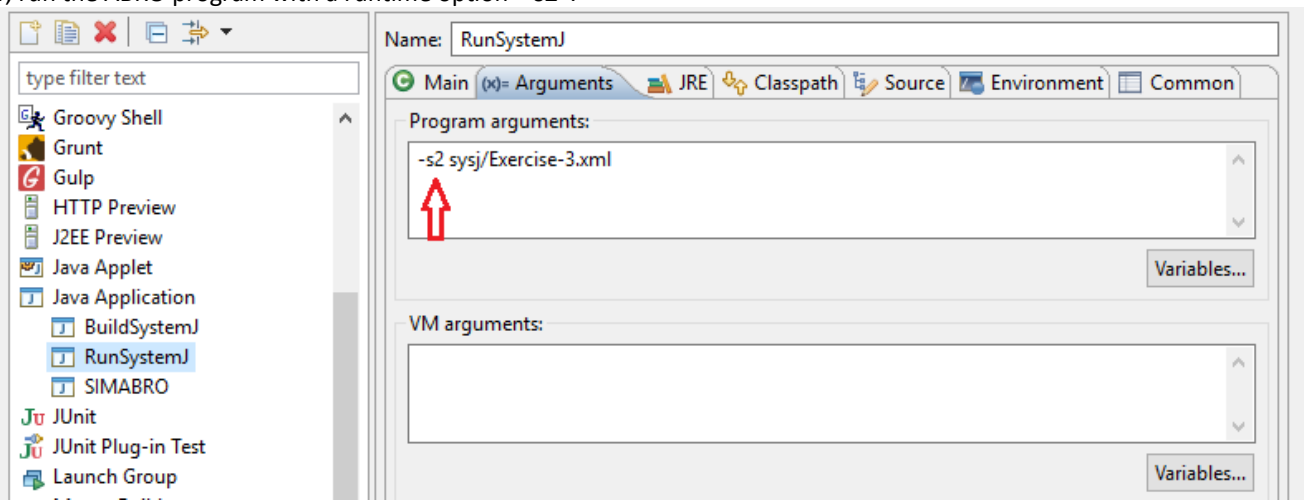
Compile and run ABRO and SIMABRO. Try to generate the output signal O by providing signals A and B from the SIMABRO.

3.1 Using the SystemJ debugger to simulate ABRO

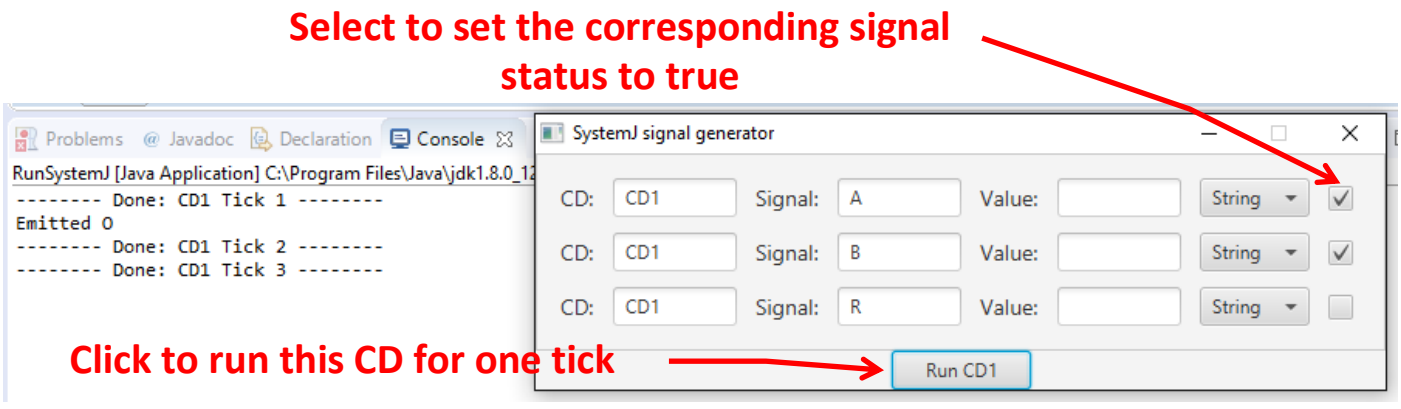
In the above example, you were already provided with a Java program, which is written specifically for simulating the ABRO example. You would need to develop additional testing program like above which can be quite a burden in some cases. Fortunately, the SystemJ runtime provides a debugging environment, which can generate input signals to clock-domains under test. You can choose which signals to be controlled manually in a debugging mode by removing the corresponding "**iSignal**" entries in the configuration file (**Exercise-3.xml**) as shown below:

```
<System xmlns="http://systemjtechnology.com">
 <SubSystem Name="ABRO_SS" Local="true">
  <ClockDomain Name="CD1" Class="ABROCD">
    <!-- In this case all the input signals will be controlled -->
    <!-- Signal O can also be removed if it is not actually received by the env. -
->
  </ClockDomain>
 </SubSystem>
</System>
```

Now, run the ABRO program with a runtime option "**-s2**":



Then the program will start with a debugging panel, which can be used to generate signals A, B, and R. Note that in a debugging mode, clock-domains run ticks once at a time when you click the "Run <CD name>" button as shown below. Select tick boxes on the right-side of the panel for signals that you would like to provide to the clock-domain in the next tick.

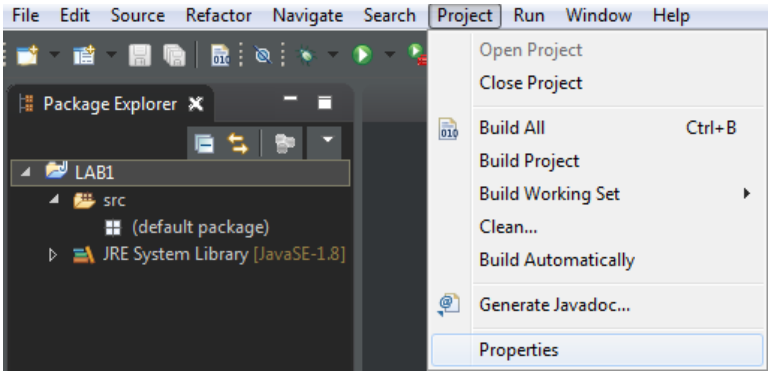


3.2 Using the sysjnetapi library

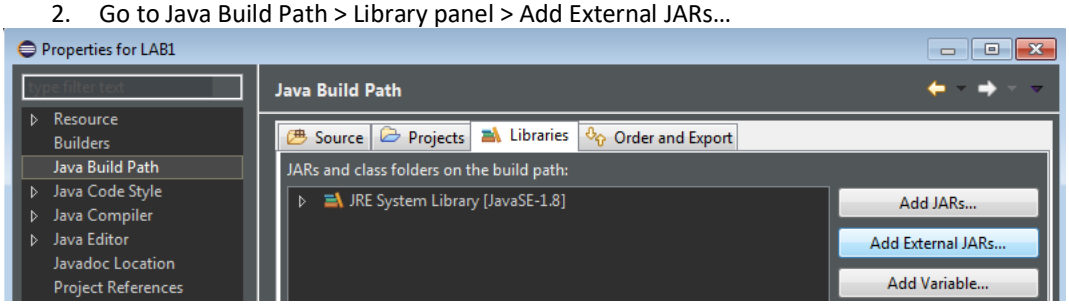
The SystemJ program can talk to other programs via I/O signals, which are usually implemented using TCP sockets. In such case, programmers need to manage TCP servers (most likely involve multithreading) and socket connections on their own. Moreover, they also need to understand data-structure of the SystemJ signal and how to initiate data exchange with the SystemJ runtime, and etc. The sysjnetapi library abstracts away such delicate details, which can introduce additional complexities to the program. You can download the library at https://davidhjp.github.io/sysjnetapi/. Run **gradlew** (Linux) or **gradlew.bat** (Windows) script to build the library jar file, which will be placed in `build/libs` directory.

Create an eclipse Java project and import the sysjnetapi library by following the steps below:

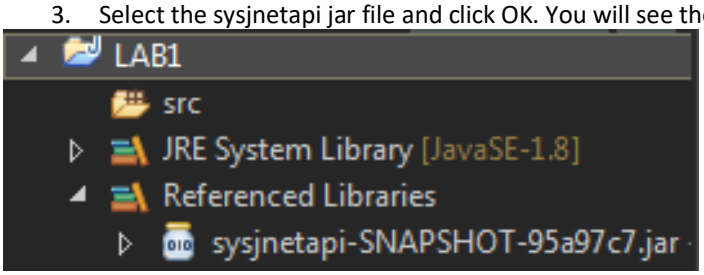
1. Select the project in the Package Explorer, and go to Project > Properties



2. Go to Java Build Path > Library panel > Add External JARs…



3. Select the sysjnetapi jar file and click OK. You will see the jar file is added as “Reference Libraries” as shown below



Exercise 4

Write a Java program which emits A, B, R in sequence to simulate the ABRO program. **Hint**: To provide the input signals to the program use `com.systemj.netapi.TCPSender`. Javadocs for the sysjnetapi can be found at http://hjparker.github.io/sysjnetapi

Now revert the **Exercise-3.xml** (ABRO) file to the original version which contains all the “**iSignals**” entries with TCPSender/Receivers. Then run **ABRO** followed by your Java program that uses sysjnetapi.

4 Writing GALS programs

4.1 Using channels

SystemJ programs can have multiple clock-domains which run asynchronously with each other. Clock-domains synchronize and exchange data through ‘blocking’ channels. The following program (**Exercise-5.sysj**) demonstrates the use of channels in SystemJ program:

```
CD1(output String channel CH;)->{
 send CH(“From CD1”);
 System.out.println(“Sent a message”);
}

CD2(input String channel CH;)->{
 receive CH;
 System.out.println(“Got a message ”+(String)#CH);
}
```

The corresponding LCF file, **Exercise-5.xml**, is also shown as follows:

```
<System xmlns="http://systemjtechnology.com">
 <SubSystem Name="CHAN" Local="true">
  <ClockDomain Name="MyCD1" Class="CD1">
    <oChannel Name="CH" To="MyCD2.CH" />
  </ClockDomain>
  <ClockDomain Name="MyCD2" Class="CD2">
    <iChannel Name="CH" From="MyCD1.CH" />
  </ClockDomain>
 </SubSystem>
</System>
```

Here, the LCF shows two new entries: '**iChannel**' and '**oChannel**' for input and output channels respectively. Both channels are called '**CH**' in the program (for simplicity, although they don't necessarily need to have the same name), so the value of '**Name**' should be also CH. **To="CD2.CH"** of **oChannel** indicates that any data sent over the output channel CH in MyCD1 is delivered *to* the input channel CH in MyCD2. Similarly **From="MyCD1.CH"** of **iChannel** denotes the data coming from the input channel CH is *from* the CH of the clock-domain MyCD1.

Exercise 5

Compile and run **Exercise-5.sysj**. You should be able to see the string "From CD1" printed from CD2.

4.2 PCABRO

PCABRO is an extended version of ABRO, where there two ABRO clock-domains (PABRO and CABRO) which have been enhanced to act as a producer and a consumer clock-domain respectively. These two clock-domains run concurrently with a third clock-domain, BUFFER, which acts as the intermediary between the two. The BUFFER clock-domain implements a circular buffer. The PABRO clock-domain produces a series of Fibonacci numbers and passes them, one by one, into the circular buffer using a channel called producerChannel. The CABRO clock-domain receives the data from the BUFFER clock-domain via the channel called consumerChannel. The BUFFER clock-domain runs two reactions concurrently: one receives data from the PABRO clock-domain via producerChannel, while the other sends the data onwards to the CABRO clock-domain via consumerChannel. The program structure is shown in Figure 4.1.

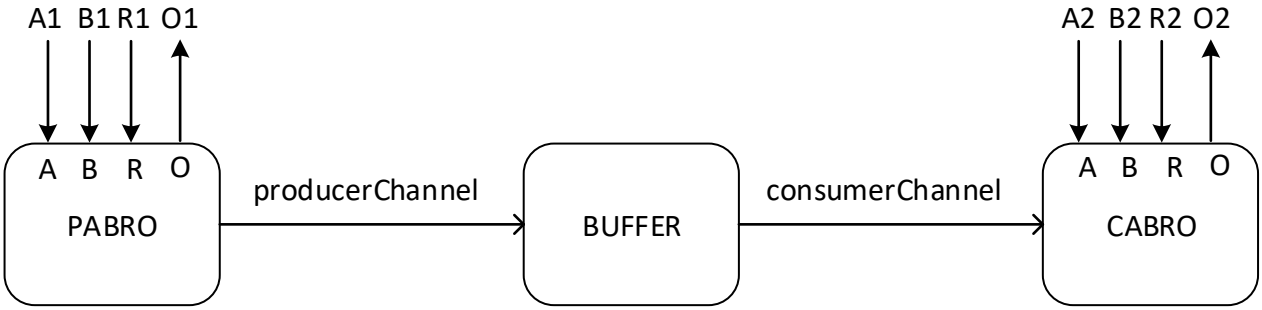


Figure 4.1 An overview of the PCABRO program

The PABRO clock-domain produces one number at a time when signals A1 and B1 are supplied from the environment. This number is then sent to the BUFFER clock-domain via producerChannel. Once the number is sent, PABRO can only produce the next number after receiving R1 from the environment. CABRO retrieves the next Fibonacci number from the BUFFER clock-domain via consumerChannel when the environment provides both signals A2 and B2. Similar to PABRO, CABRO can only retrieve the next number after it gets signal R2 from the environment.

Exercise 6

In this example, you are given only a partially implemented PCABRO which has several lines of code missing in the BUFFER clock-domain. Fill in the missing lines and then run the complete PCABRO program.

The BUFFER clock-domain should be able to:

- Receive the next Fibonacci sequence from the PABRO CD via producerChannel.
- Send the next Fibonacci sequence to the CABRO CD via consumerChannel.
- Buffer the numbers from the Fibonacci sequence sent from the PABRO CD even if the CABRO CD is not able to receive it due to differences in the execution speed of the two clock domains.

In **Exercise-6.sysj**, there are 'TODO' comments where you have to write additional SystemJ code.

Your task is

1. ***Fill in the missing lines in Exercise-6.sysj*** e.g.:

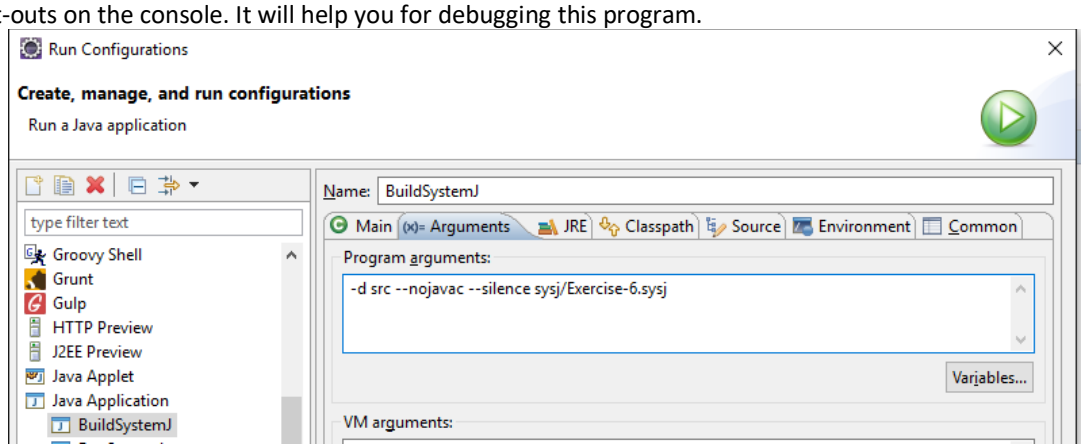
```
/* Task 1: receive from producerChannel */
// Write a channel receive statement
```

2. ***Complete the LCF file (Exercise-6.xml) for the BUFFER clock-domain***

Note: Java objects are local to the reaction in which they are declared. Make sure you read the SystemJ delayed semantics appendix on Canvas to prevent any misunderstanding with regards to sharing variables across reactions.

You are given buffer.Buffer class file which can be used in implementing the BUFFER clock-domain (data-part). You are given a simulator program called "**SIMPCABRO.java"** which generates input signals to the PABRO and CABRO clock-domains.

Compile and run Exercise-6.sysj. Note that by giving an option **--silence** to the compiler (see below), you can suppress signal emission print-outs on the console. It will help you for debugging this program.



Finally, compile and run **SIMPCABRO.java** as a separate Java application.

This will open a GUI similar to the one for the ABRO example. Test your design by generating inputs to the PABRO and CABRO clock-domains.

4.3 Distributing clock-domains to different machines

In the prior PCABRO example, all clock-domains were run on a single machine (JVM). However, the SystemJ RTS allows dividing the program into one or more subsystems which can be distributed across different machines while preserving the overall behaviour of the program! Here, we are going to demonstrate this by dividing the PCABRO program into two '**subsystems',** one running the PABRO and CABRO clock-domains, and the other one running the BUFFER clock-domain. In this case, channels between clock-domains in different subsystems are implemented on an abstraction called **Link**. Link is used to define how two subsystems communicate each with the other, e.g. by using some kind of network. As the notion of a subsystem is closely related to the machine, real or virtual, on which the subsystem runs, the link is also associated with the pair of machines, real and/or virtual, which enable data exchange. Obviously, this communication must be supported within the SystemJ RTS as well, because the data sent via channels must be in reality exchanged between the source and destination machines.

In this lab, from practical reasons, we are going to implement two subsystems on different virtual machines (JVMs); thus the link will have to be established between two virtual machines. This will be done by using TCP/IP.

Each subsystem is specified with a local configuration file (LCF), using XML, that lists all clock-domains that belong to the subsystem, all subsystems with which this subsystem communicates, and all links which are used to enable communication of this subsystem with other subsystems in the SystemJ program. The LCF is used by the RTS in initialisation of a subsystem on the machine which it will run.

Using the previous PCABRO example, we create two subsystems as explained above by specifying two Local Configuration Files (LCFs): **Exercise-7-pcabro1.xml** and **Exercise-7-pcabro2.xml** as shown in Figure 4.2 and Figure 4.3, respectively. Note that some parts of the file are omitted for brevity. Here, a link between subsystem 1 (SS1) and subsystem 2 (SS2) is specified using an element called **Link** as shown in lines 3-6 of Figure 4.2 and 4.3. We specify a type of the link as "Destination" when it is implemented using TCP socket (i.e. class TCPIPInterface) because destination address is used to establish a TCP connection. The **Args** attributes (lines 4-5) are in the form of `<IP>:<Port>`, which specifies a socket address where the corresponding subsystem can be reached. Unlike signals, **oChannel** and **iChannel** elements do not require additional attributes for **IP addresses**

and **Ports** (as shown in Figure 4.2 and Figure 4.3) since RTS automatically resolves them using the Links declared in the configuration file.

An attribute **Local="true"** for the subsystem **SS1** (line 9 in Figure 4.2) indicates that one RTS will start a subsystem consisting of the PABRO and CABRO clock-domains. Run the program via "**RunSystemJ**" with an argument **Exercise-7-pcabro1.xml**.

Likewise, the other RTS starts a subsystem having a Buffer clock-domain via "**RunSystemJ**" with an argument **Exercise-7-pcabro2.xml**.

Exercise 7

Compile and run the distributed PCABRO by running two SystemJ program instances (**RunSystemJ**) with LCF files Exercise-7-pcabro1.xml and Exercise-7-pcabro2.xml. Simulate the program using the SIMPCABRO.

```
1  <System xmlns="http://systemjtechnology.com">
2   <Interconnection>
3    <Link Type="Destination">
4      <Interface SubSystem="SS1" Class="com.systemj.ipc.TCPIPInterface" Args="127.0.0.1:1112"/>
5      <Interface SubSystem="SS2" Class="com.systemj.ipc.TCPIPInterface" Args="127.0.0.1:1113"/>
6    </Link>
7   </Interconnection>
8
9    <SubSystem Name="SS1" Local="true">
10       <ClockDomain Name="PabroCD" Class="PabroCD">
11          ... omitted ...
12        <oChannel Name="producerChannel" To="BufferCD.producerChannel" />
13       </ClockDomain>
14       <ClockDomain Name="CabroCD" Class="CabroCD">
15         ... omitted ...
16        <iChannel Name="consumerChannel" From="BufferCD.consumerChannel" />
17      </ClockDomain>
18   <SubSystem>
19   <SubSystem Name="SS2”>
20       <ClockDomain Name="BufferCD" Class="BufferCD">
21          ... omitted ...
22       </ClockDomain>
23   </SubSystem>
24 </System>
```

Figure 4.2 Exercise-7-pcabro1.xml file

```
1  <System xmlns="http://systemjtechnology.com">
2   <Interconnection>
3    <Link Type="Destination">
4      <Interface SubSystem="SS1" Class="com.systemj.ipc.TCPIPInterface" Args="127.0.0.1:1112"/>
5      <Interface SubSystem="SS2" Class="com.systemj.ipc.TCPIPInterface" Args="127.0.0.1:1113"/>
6    </Link>
7   </Interconnection>
8
9    <SubSystem Name="SS1">
10       <ClockDomain Name="PabroCD" Class="PabroCD">
11          ... omitted ...
12          <oChannel Name="producerChannel" To="BufferCD.producerChannel" />
13       </ClockDomain>
14       <ClockDomain Name="CabroCD" Class="CabroCD">
15         ... omitted ...
16          <iChannel Name="consumerChannel" From="BufferCD.consumerChannel" />
17      </ClockDomain>
18   <SubSystem>
19   <SubSystem Name="SS2” Local=”true”>
20       <ClockDomain Name="BufferCD" Class="BufferCD">
21          ... omitted ...
22       </ClockDomain>
23   </SubSystem>
24 </System>
```

Figure 4.3 Exercise-7-pcabro2.xml file

4.4 Exchange data with environment using JSON data

SystemJ clock-domains can exchange data in JSON format with external programs. Consider the following SystemJ code and LCF file.

```
CD1(input Integer signal A; output Integer signal O1,O2;)->{
    int counter;
    loop {
        await(A);
        pause;
        emit O1(counter++);
        pause;
        emit O2(counter++);
        pause;
    }
}

CD2(input Integer signal I;)->{
    loop{
        await(I);
        System.out.println("Received "+#I);
    }
}
```

Figure 4.4 SystemJ program for Exercise 7

```
<System xmlns="http://systemjtechnology.com">
  <SubSystem Name="SS1" Local="true">
    <ClockDomain Name="CD1" Class="CD1">
      <iSignal Name="A" Class="com.systemj.ipc.InputJSON" IP="127.0.0.1"   Port="100" Type="java.lang.Integer"/>
      <oSignal Name="O1" Class="com.systemj.ipc.OutputJSON" IP="127.0.0.1" Port="200" />
      <oSignal Name="O2" Class="com.systemj.ipc.OutputJSON" IP="127.0.0.1" Port="100" To="CD2.I"/>
    </ClockDomain>
    <ClockDomain Name="CD2" Class="CD2">
      <iSignal Name="I" Class="com.systemj.ipc.InputJSON" IP="127.0.0.1" Port="100" Type="java.lang.Integer"/>
    </ClockDomain>
  </SubSystem>
</System>
```

Figure 4.5 An XML file for Exercise 7

As shown in Figure 4.4, the program consists of two clock-domains, CD1 and CD2. CD1 captures signal A from environment then emits signals O1 and O2 with a counter value incremented by one. O1 is transmitted to the environment, while O2 is delivered to CD2. The configuration file shown in Figure 4.5 uses IPC classes com.systemj.ipc.InputJSON and com.systemj.ipc.OutputJSON for input and output signals, respectively. For example, external environment, for example a Java program, can provide signal A for CD1 by transmitting JSON data as shown in Figure 4.6, where the keys are:

1. `name` is the name of the input signal (type String).
2. `cd` is the name of the clock-domain which has this input signal (type String).
3. `status` is a status of the signal (type Boolean).
4. `value` is a value of the signal (type String).

```
{
    "name": "A",
    "cd": "CD1",
    "status": true,
    "value": "123"
}
```

Figure 4.6 JSON data that is received by the SystemJ runtime.

An attribute `Type` for the iSignals in Figure 4.5 converts the received `value` to a Java object instance by invoking a constructor of the specified class, for example `new java.lang.Integer("123")` in this case. When a clock-domain emits signal using OutputJSON, values for `name` and `cd` are automatically set to the names of the emitting signal and the corresponding clock-domain. For example, signal O1 shown in Figure 4.5 will set JSON values for `name` and `cd` to "O1" and "CD1", respectively. A user can override this behaviour by setting "To" attribute for the iSignals in the LCF file. For example, the attribute `To="CD2.I"` for signal O2 will have an effect setting `name` and `cd` values to "CD2" and "I", respectively.

Exercise 8

Write a Java program which communicates with CD1 and CD2 in Figure 4.4 via signal A and O1. **Hint:** You need to create a TCP server that listens on port 200 and a TCP client that connects to port 100 (see the LCF in Figure 4.5). Use created TCP sockets to transmit and receive JSON data.

Note that you will probably find it convenient to use a JSON library such as GSON (https://github.com/google/gson). It can be downloaded from http://repo1.maven.org/maven2/com/google/code/gson/gson/2.8.1/.